Project

Mobile Application Development

dr inż. Radosław Maciaszczyk
Faculty of Computer Science
West Pomeranian University of Technology, Szczecin

Project – Class 3

Deadline: 28 April 2026

Define the **data model** and **API contracts** of your application, lay out the **Clean-Architecture skeleton** in Android Studio, and push the first commit to your GitHub repository.

Task Requirements

1. **Specify what data will be stored**
   List every piece of data the application needs to persist or transfer:
   - data fetched from **remote APIs** (JSON/REST responses),
   - data stored **locally** (Room database, DataStore, files),
   - any **transient state** kept only in memory (ViewModel).

   For each item state the **storage location**, **lifetime**, and **reason** for storing it.

2. **Build data classes**
   Write the Kotlin data classes (or sealed classes) that represent:
   - **Domain models** – clean objects used across the app (no framework annotations, no JSON keys),
   - **Data Transfer Objects (DTOs)** – mirror the JSON structure received from the API (annotated with `@SerializedName` or `@Json`),
   - **Room Entities** – annotated with `@Entity`, `@PrimaryKey`, etc.

   Include at least the **field name**, **type**, and a short comment for each non-obvious field.

3. **Define the API**
   Specify the interfaces the application will consume or expose:
   - **Retrofit service interface(s)** – one method per endpoint, with full URL path, HTTP verb, query parameters, and return type (`Response<Dto>` or `Flow<Dto>`),
   - **Room DAO interface(s)** – one method per database operation (insert, query, delete), with appropriate suspend / Flow return types,

- **Repository interface(s)** – the contract between the domain layer and the data layer (uses domain models, not DTOs or entities).

4. **Create the application skeleton**
   Set up an Android Studio project with the following **package structure**:

   - `ui/` – Composable screens, ViewModels, UI state classes,
   - `data/` – repository implementations, Room database, Retrofit service, DTOs, mappers,
   - `domain/` – domain models, repository interfaces, use cases (if applicable).

   Every package should contain at least a **stub file** (*empty class / object / interface*) so that the project compiles. Add all required **Gradle dependencies** (Compose, Room, Retrofit, Hilt, etc.) to `build.gradle.kts`.

5. **Create repository and upload the first commit**

   - Create a **public GitHub repository** named after your application (e.g. `WeatherNow`).
   - Add a `README.md` with: project name, short description, complexity level, and build instructions.
   - Add a `.gitignore` appropriate for Android/Kotlin projects (use the *Android* template from `https://gitignore.io`).
   - Push the skeleton with the commit message: `feat: initial application skeleton`.
   - Submit the **repository URL** together with this document.

Deliverables

- A **PDF file** (this document filled in) containing tasks 1–3.
- A link to the **GitHub repository** with the skeleton (task 4–5).

> **Note:** The data model and API interfaces defined here *will* evolve – that is expected. The goal at this stage is to demonstrate a clear understanding of *what* data the application needs and *how* it will be accessed.

Project – Class 3

Sample description

Application: WeatherNow (Medium level – continued from Classes 1 and 2)

1. Data specification

Data	Location	Lifetime	Reason
Current weather	API (OpenWeatherMap)	Session	Display live conditions
5-day forecast	API (OpenWeatherMap)	Session	Show upcoming weather
Search history (last 10 cities)	Room DB	Persistent	Quick re-access without re-typing
Favourite locations	Room DB	Persistent	User-curated shortlist on Home
Active UI state (selected city, loading flag, error message)	ViewModel	Screen lifetime	Survive rotation, drive UI

2. Data classes

2a. Domain models (domain/model/)

```kotlin
// domain/model/CurrentWeather.kt
data class CurrentWeather(
    val cityName: String,
    val countryCode: String,
    val temperatureCelsius: Double,
    val feelsLikeCelsius: Double,
    val humidity: Int,          // percent
    val windSpeedMs: Double,    // metres per second
    val conditionText: String,  // e.g. "Clear sky"
    val iconCode: String,       // e.g. "01d"  -- used to build icon
       URL
    val latitude: Double,
    val longitude: Double
)
```

Listing 1: CurrentWeather.kt – domain model

```kotlin
// domain/model/ForecastItem.kt
data class ForecastItem(
    val dateTimeEpoch: Long,    // Unix timestamp (UTC)
```

```
    val temperatureCelsius: Double,
    val conditionText: String,
    val iconCode: String
)
```

Listing 2: ForecastItem.kt – domain model

2b. DTOs (API responses) (data/remote/dto/)

```
// data/remote/dto/WeatherResponseDto.kt
data class WeatherResponseDto(
    @SerializedName("name")    val cityName: String,
    @SerializedName("sys")     val sys: SysDto,
    @SerializedName("main")    val main: MainDto,
    @SerializedName("wind")    val wind: WindDto,
    @SerializedName("weather") val weather: List<WeatherConditionDto
       >,
    @SerializedName("coord")   val coord: CoordDto
)

data class SysDto(
    @SerializedName("country") val countryCode: String
)

data class MainDto(
    @SerializedName("temp")       val temp: Double,
    @SerializedName("feels_like") val feelsLike: Double,
    @SerializedName("humidity")   val humidity: Int
)

data class WindDto(
    @SerializedName("speed") val speed: Double
)

data class WeatherConditionDto(
    @SerializedName("description") val description: String,
    @SerializedName("icon")        val icon: String
)

data class CoordDto(
    @SerializedName("lat") val lat: Double,
    @SerializedName("lon") val lon: Double
)
```

Listing 3: WeatherResponseDto.kt – mirrors OpenWeatherMap /weather JSON

```
// data/remote/dto/ForecastResponseDto.kt
data class ForecastResponseDto(
    @SerializedName("list") val items: List<ForecastItemDto>
)
```

```
data class ForecastItemDto(
    @SerializedName("dt")      val dt: Long,
    @SerializedName("main")    val main: MainDto,
    @SerializedName("weather") val weather: List<WeatherConditionDto>
)
```

Listing 4: ForecastResponseDto.kt – mirrors OpenWeatherMap /forecast JSON

2c. Room Entities (data/local/entity/)

```
// data/local/entity/LocationHistoryEntity.kt
@Entity(tableName = "location_history")
data class LocationHistoryEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val cityName: String,
    val countryCode: String,
    val latitude: Double,
    val longitude: Double,
    val searchedAtEpoch: Long   // Unix timestamp of last search
)
```

Listing 5: LocationHistoryEntity.kt – search history

```
// data/local/entity/FavouriteEntity.kt
@Entity(tableName = "favourites")
data class FavouriteEntity(
    @PrimaryKey val cityName: String,   // city name used as unique
        key
    val countryCode: String,
    val latitude: Double,
    val longitude: Double
)
```

Listing 6: FavouriteEntity.kt – favourite locations

3. API definition

3a. Retrofit service interface (data/remote/)

```
// data/remote/WeatherApiService.kt
interface WeatherApiService {

    @GET("weather")
    suspend fun getCurrentWeather(
        @Query("q")     cityName: String,
        @Query("lat")   lat: Double? = null,
        @Query("lon")   lon: Double? = null,
        @Query("units") units: String = "metric",
        @Query("appid") apiKey: String = BuildConfig.OWM_API_KEY
    ): Response<WeatherResponseDto>
```

```
    @GET("forecast")
    suspend fun getForecast(
        @Query("q")     cityName: String,
        @Query("lat")   lat: Double? = null,
        @Query("lon")   lon: Double? = null,
        @Query("units") units: String = "metric",
        @Query("cnt")   count: Int = 40,        // 5 days x 8 slots/
           day
        @Query("appid") apiKey: String = BuildConfig.OWM_API_KEY
    ): Response<ForecastResponseDto>
}
```

Listing 7: WeatherApiService.kt

3b. Room DAO interfaces (data/local/dao/)

```
// data/local/dao/LocationHistoryDao.kt
@Dao
interface LocationHistoryDao {
    @Query("SELECT * FROM location_history ORDER BY searchedAtEpoch
       DESC LIMIT 10")
    fun getHistory(): Flow<List<LocationHistoryEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(entity: LocationHistoryEntity)

    @Query("DELETE FROM location_history WHERE id NOT IN " +
           "(SELECT id FROM location_history ORDER BY searchedAtEpoch
               DESC LIMIT 10)")
    suspend fun pruneOldEntries()
}
```

Listing 8: LocationHistoryDao.kt

```
// data/local/dao/FavouriteDao.kt
@Dao
interface FavouriteDao {
    @Query("SELECT * FROM favourites ORDER BY cityName ASC")
    fun getFavourites(): Flow<List<FavouriteEntity>>

    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun addFavourite(entity: FavouriteEntity)

    @Delete
    suspend fun removeFavourite(entity: FavouriteEntity)

    @Query("SELECT EXISTS(SELECT 1 FROM favourites WHERE cityName = :
       cityName)")
    suspend fun isFavourite(cityName: String): Boolean
}
```

Listing 9: FavouriteDao.kt

3c. Repository interfaces (domain/repository/)

```
// domain/repository/WeatherRepository.kt
interface WeatherRepository {
    suspend fun getCurrentWeather(cityName: String): Result<
       CurrentWeather>
    suspend fun getCurrentWeatherByCoords(lat: Double, lon: Double):
       Result<CurrentWeather>
    suspend fun getForecast(cityName: String): Result<List<
       ForecastItem>>
}
```

Listing 10: WeatherRepository.kt – domain contract

```
// domain/repository/LocationRepository.kt
interface LocationRepository {
    fun getHistory(): Flow<List<LocationHistoryEntity>>
    suspend fun saveToHistory(weather: CurrentWeather)

    fun getFavourites(): Flow<List<FavouriteEntity>>
    suspend fun addFavourite(weather: CurrentWeather)
    suspend fun removeFavourite(cityName: String)
    suspend fun isFavourite(cityName: String): Boolean
}
```

Listing 11: LocationRepository.kt – domain contract

4. Application skeleton structure

```
com.example.weathernow/
|
+-- di/                              # Hilt modules
|   +-- AppModule.kt                 # provides Retrofit, Room, OkHttp
|   +-- RepositoryModule.kt          # binds interfaces to implementations
|
+-- domain/                          # pure Kotlin -- no Android dependencie
|   +-- model/
|   |   +-- CurrentWeather.kt
|   |   +-- ForecastItem.kt
|   +-- repository/
|       +-- WeatherRepository.kt
|       +-- LocationRepository.kt
|
+-- data/                            # framework-dependent implementations
|   +-- remote/
|   |   +-- dto/
|   |   |   +-- WeatherResponseDto.kt
|   |   |   +-- ForecastResponseDto.kt
|   |   +-- WeatherApiService.kt
|   |   +-- mapper/
|   |       +-- WeatherMapper.kt
|   +-- local/
|   |   +-- entity/
|   |   |   +-- LocationHistoryEntity.kt
|   |   |   +-- FavouriteEntity.kt
|   |   +-- dao/
|   |   |   +-- LocationHistoryDao.kt
|   |   |   +-- FavouriteDao.kt
|   |   +-- AppDatabase.kt
|   +-- repository/
|       +-- WeatherRepositoryImpl.kt
|       +-- LocationRepositoryImpl.kt
|
+-- ui/                              # Jetpack Compose screens + ViewModels
    +-- home/
    |   +-- HomeScreen.kt
    |   +-- HomeViewModel.kt
    |   +-- HomeUiState.kt
    +-- search/
    |   +-- SearchScreen.kt
    |   +-- SearchViewModel.kt
    +-- detail/
    |   +-- DetailScreen.kt
    |   +-- DetailViewModel.kt
    +-- history/
```

```
|   +-- HistoryScreen.kt
|   +-- HistoryViewModel.kt
+-- favourites/
|   +-- FavouritesScreen.kt
|   +-- FavouritesViewModel.kt
+-- navigation/
    +-- AppNavGraph.kt
```

Key Gradle dependencies (`build.gradle.kts – app module`)

```
// Jetpack Compose BOM
implementation(platform("androidx.compose:compose-bom:2024.09.00"))
implementation("androidx.compose.ui:ui")
implementation("androidx.compose.material3:material3")
implementation("androidx.activity:activity-compose:1.9.2")

// Navigation Compose
implementation("androidx.navigation:navigation-compose:2.8.0")

// ViewModel + Lifecycle
implementation("androidx.lifecycle:lifecycle-viewmodel-compose
   :2.8.4")

// Room
implementation("androidx.room:room-runtime:2.6.1")
implementation("androidx.room:room-ktx:2.6.1")
ksp("androidx.room:room-compiler:2.6.1")

// Retrofit + Gson
implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.retrofit2:converter-gson:2.11.0")
implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

// Hilt
implementation("com.google.dagger:hilt-android:2.51.1")
ksp("com.google.dagger:hilt-compiler:2.51.1")
implementation("androidx.hilt:hilt-navigation-compose:1.2.0")

// Google Maps Compose
implementation("com.google.maps.android:maps-compose:4.4.1")
implementation("com.google.android.gms:play-services-location
   :21.3.0")

// Coil (image loading for weather icons)
implementation("io.coil-kt:coil-compose:2.7.0")
```

Listing 12: build.gradle.kts (dependencies block – excerpt)

Reference – Android Architecture Samples: The official Google repository `https://github.com/android/architecture-samples` (branch `main`, last updated April 2026) uses the identical stack and package structure (Compose, Hilt, Room, Flow, Navigation Compose). You may use it as a **structural reference**, but you must implement your own domain model, data layer, and UI screens – do not submit a re-skinned TODO app.

Tip: Store your API key in `local.properties` (never commit this file – it is already in the Android `.gitignore`): `OWM_API_KEY=your_key_here`. Read it in `build.gradle.kts` via `buildConfigField("String", "OWM_API_KEY", ...)` and enable `buildConfig = true` in the `android { buildFeatures { } }` block.

5. Repository and first commit

Item	Details
Repository URL	`https://github.com/<username>/WeatherNow`
Visibility	Public
First commit	`feat: initial application skeleton`
`.gitignore`	Android template (generated via `https://gitignore.io`)
`README.md`	Project name, description, complexity level (Medium), build steps, API key setup, screenshots placeholder

Checklist before submitting:

- ☐ Project compiles without errors in Android Studio.
- ☐ All three packages (`ui/`, `data/`, `domain/`) exist and contain at least stub files.
- ☐ All Gradle dependencies listed above are present in `build.gradle.kts`.
- ☐ `local.properties` is **not** tracked by Git.
- ☐ The GitHub repository is public and the first commit is visible.
- ☐ The repository URL is included in the submitted PDF.